DEVOPS PROJECT · 2026

Container Network Policy Generator

Automated Kubernetes NetworkPolicy manifests via a secure, containerised RESTful microservice.
Zero Trust by default · Infrastructure as Code · Full CI/CD pipeline.

FastAPI | Pydantic | Docker | Kubernetes | GitHub Actions | Python 3.11 | pytest

Arya Duttagupta · MIT License · github.com/unpetrifiedstone2005/Devops_project

TABLE OF CONTENTS

What's Inside

01 **Project Overview**
Problem · Solution · Impact

02 **System Architecture**
4-Layer design · Logic flow

03 **Core Source Code**
main.py · logic.py · AppSpec model

04 **API Endpoints & Usage**
/generate · /health · Swagger UI

05 **Input Schema (IaC)**
app.json · field definitions

06 **Containerisation**
Dockerfile · Docker Compose

07 **CI/CD Pipeline**
GitHub Actions · 7 stages

08 **Testing Strategy**
Unit tests · Integration tests

09 **Security Design**
Zero Trust · Auth · Secrets

10 **Monitoring & Observability**
Health probes · Structured logging

11 **Tech Stack**
Full dependency overview

12 **Repository Structure**
Directory layout · file roles

13 **How to Run**
Local · Docker · Curl examples

14 **Project Summary**
Key deliverables & next steps

01 / PROJECT OVERVIEW

The Network Policy Generator evolved from a CLI utility into a Secure RESTful Microservice — automating Kubernetes NetworkPolicy YAML from a simple JSON spec.

PROBLEM

- Kubernetes NetworkPolicy YAML is verbose and error-prone
- Manual authoring doesn't scale across large microservice estates
- Mistakes lead to over-permissive rules and security breaches
- No standard IaC interface for network policy creation

SOLUTION

- REST API accepts structured JSON, returns ready-to-deploy YAML
- Pydantic validation ensures only correct schemas are processed
- Single source of truth: app.json defines all service dependencies
- Containerised for portability — runs identically everywhere

IMPACT

- Zero Trust enforced by default — deny-all baseline
- Policy updates require only a JSON edit, not YAML expertise
- Plugs directly into CI/CD pipelines for automated compliance
- Reduces human error in security-critical network configuration

02 / SYSTEM ARCHITECTURE

1 API Layer

FastAPI + Uvicorn

- Routing to /generate & /health
- X-API-KEY header auth
- Async request handling
- OpenAPI schema generation

2 Validation

Pydantic Models

- AppSpec model enforcement
- Type checking (str, int, list)
- 422 error on bad payload
- Auto-generates Swagger docs

3 Core Engine

logic.py

- AppSpec → K8s YAML mapping
- podSelector from labels
- Ingress & egress policy build
- networking.k8s.io/v1 spec

4 Infra Layer

Docker & Compose

- python:3.11-slim base image
- Port mapping 8000:8000
- Env var config injection
- Bridge network isolation

REQUEST LIFECYCLE

Client sends POST + X-API-KEY › **Auth check** › **Pydantic validates JSON** › **Logic engine transforms** › **YAML returned as JSON**

03 / CORE SOURCE CODE

src/main.py — FastAPI Application Entry Point

```python
import os
from fastapi import FastAPI, Header, HTTPException
from src.logic import AppSpec, generate_policy

app = FastAPI()
API_KEY = os.getenv("API_KEY", "devops-internal-secret")

@app.get("/health")
def health():
    return {"status": "healthy"}

@app.post("/generate")
def api_generate(spec: AppSpec,
                 x_api_key: str = Header(None)):
    if x_api_key != API_KEY:
        raise HTTPException(403, "Unauthorized")
    return {"yaml": generate_policy(spec)}
```

src/logic.py — NetworkPolicy Generation Engine

```python
import yaml
from pydantic import BaseModel
from typing import List, Dict

class AppSpec(BaseModel):
    name: str
    namespace: str = "default"
    labels: Dict[str, str]
    ingress_sources: List[Dict] = []
    egress_destinations: List[Dict] = []

def generate_policy(spec: AppSpec) -> dict:
    return {
      "apiVersion": "networking.k8s.io/v1",
      "kind": "NetworkPolicy",
      "spec": {
        "podSelector": {"matchLabels": spec.labels},
        "ingress": [ ... for src in ingress_sources ],
        "egress":  [ ... for dst in egress_destinations ]
    }
```

04 / API ENDPOINTS & USAGE

GET

Liveness & Readiness Probe

Returns {"status": "healthy"} — used by Docker healthcheck and Kubernetes orchestrators for liveness and readiness probes. No authentication required.

Response: 200 OK · No auth needed · Used by Docker & K8s

POST

NetworkPolicy Manifest Generator

Accepts an AppSpec JSON body. Requires X-API-KEY header. Validates the payload via Pydantic, runs the generation engine, and returns the Kubernetes NetworkPolicy YAML manifest.

200 OK — YAML returned

403 Unauthorized

422 Validation Error

Interactive Swagger UI — http://127.0.0.1:8000/docs

FastAPI auto-generates a full OpenAPI 3.0 specification and interactive Swagger UI. You can:

- Click Authorize and enter the API key to authenticate
- Expand POST /generate and click Try it out
- Paste an AppSpec JSON body and click Execute
- View the full NetworkPolicy YAML directly in the browser response

05 / INPUT SCHEMA — INFRASTRUCTURE AS CODE

The app.json file is the IaC source of truth. Every network rule is defined here — no YAML expertise required.

FIELD	TYPE	REQUIRED	DESCRIPTION
name	String	Required	Unique name of the application (used in policy metadata.name)
namespace	String	Optional	Kubernetes namespace — defaults to "default" if not specified
labels	Object	Required	Key-value labels used for podSelector matchLabels to target pods
ingress_sources	Array	Optional	List of {app_name, port} — services ALLOWED to send traffic TO this app
egress_destinations	Array	Optional	List of {app_name, port} — services this app is ALLOWED to talk TO

Sample app.json (production/kubernetes/app.json)

```
{ "name": "backend-api",  "namespace": "production",  "labels": {"app": "backend-server"},
  "ingress_sources":      [{"app_name": "frontend-ui", "port": 80}, {"app_name": "admin-panel", "port": 443}],
  "egress_destinations":  [{"app_name": "mongodb-db", "port": 27017}, {"app_name": "redis-cache", "port": 6379}] }
```

06 / CONTAINERISATION — DOCKERFILE & COMPOSE

infrastructure/docker/Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

RUN apt-get update && apt-get install -y curl \
    && rm -rf /var/lib/apt/lists/*

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

ENV PYTHONPATH=/app

HEALTHCHECK --interval=30s --timeout=10s \
  CMD curl -f http://localhost:8000/health

EXPOSE 8000
CMD ["uvicorn", "src.main:app",
     "--host", "0.0.0.0", "--port", "8000"]
```

infrastructure/docker/docker-compose.yml

```yaml
version: '3.8'

services:
  netgen-api:
    build:
      context: ../../
      dockerfile: infrastructure/docker/Dockerfile
    container_name: netgen_service
    ports:
      - "8000:8000"
    environment:
      - API_KEY=devops-internal-secret
      - ENV=production
    healthcheck:
      test: ["CMD","curl","-f","http://localhost:8000/health"]
      interval: 30s
      timeout: 10s
      retries: 3
    restart: unless-stopped
    networks: [netgen_network]
```

07 / CI/CD PIPELINE — GITHUB ACTIONS

Triggered on every push and pull_request — fully automated quality gate before any code reaches main.

1

Checkout Code

actions/checkout@v3
Fetches the latest source tree from the repository.

2

Setup Python

actions/setup-python@v4
python-version: 3.11 — consistent runtime.

3

Install Deps

pip install -r requirements.txt
+ flake8, pytest, httpx (API testing).

ci-cd.yml trigger

```
name: DevOps CI Pipeline

on: [push, pull_request]

jobs:
  test-and-lint:
    runs-on: ubuntu-latest
```

5

Execute Tests

python -m pytest src/tests/
Unit + integration tests for logic & API.

6

Docker Build

docker build -t policy-api-test
Verifies the container builds cleanly.

7

Compose Validate

docker compose config
Validates orchestration syntax before deploy.

08 / TESTING STRATEGY

src/tests/test_logic.py

```python
import pytest
from src.logic import AppSpec, generate_policy

def test_policy_structure():
    """Verify generated YAML has K8s fields."""
    data = {
        "name": "test-app",
        "labels": {"app": "test-app"},
        "ingress_sources": [{"app_name": "web", "port": 80}]
    }
    spec   = AppSpec(**data)
    policy = generate_policy(spec)

    assert policy["kind"] == "NetworkPolicy"
    assert "Ingress" in policy["spec"]["policyTypes"]
    assert policy["spec"]["ingress"][0]["ports"][0]["port"] == 80

def test_invalid_data():
    """Pydantic catches missing required fields."""
    invalid = {"name": "fail-app"}  # labels missing
    with pytest.raises(Exception):
        AppSpec(**invalid)
```

Test Coverage & Strategy

Unit Test: Policy Structure

Verifies that generate_policy() returns a valid K8s NetworkPolicy dict with correct kind, policyTypes, and port mapping.

Unit Test: Schema Validation

Confirms Pydantic raises an exception when required fields (labels) are missing — preventing bad payloads from reaching the engine.

Integration Tests (httpx)

The CI pipeline installs httpx to enable async HTTP testing of the live FastAPI endpoints — verifying /generate and /health end-to-end.

CI Smoke Test

The docker build and docker compose config steps act as smoke tests — ensuring the container image and compose config are always valid.

```
$ python -m pytest src/tests/ -v
```

09 / SECURITY DESIGN

Security is a first-class concern — not an afterthought. Every decision reinforces Zero Trust and least privilege.

Zero Trust by Default

- All traffic is DENIED unless explicitly declared
- No implicit allow rules — every connection must be named
- Tool enforces least-privilege at policy generation time
- Generates policyTypes: [Ingress, Egress] always

Header-Based Authentication

- X-API-KEY header required on every POST /generate call
- Returns HTTP 403 Forbidden on missing or wrong key
- Simple, stateless check — no session overhead
- Suitable for CI/CD runner-to-service auth model

Secret Externalization

- API_KEY loaded via os.getenv() — never hardcoded
- Injected via Docker Compose environment: block
- Prod secrets injected as K8s Secrets or Vault refs
- .gitignore excludes generated *-policy.yaml artifacts

Minimal Attack Surface

- python:3.11-slim — Debian base, stripped packages
- Only curl added (for healthcheck) — no extra tools
- No SSH, no shell extras, no unnecessary services
- Internal-only deployment via bridge network

10 / MONITORING & OBSERVABILITY

Health Endpoint /health

- Returns {"status": "healthy"} on GET requests
- Used by Docker Compose healthcheck block (30s interval, 10s timeout, 3 retries)
- Kubernetes liveness & readiness probes can target this endpoint
- No auth required — accessible to orchestrators at all times
- restart: unless-stopped ensures container auto-recovery

Structured Logging

- Python built-in logging library — zero extra dependencies
- Every POST /generate logs: timestamp, source IP, app name
- Validation errors logged with full Pydantic error detail
- Auth failures logged at WARNING level for security alerting
- Compatible with log aggregators: ELK Stack, Loki, CloudWatch

Docker Compose healthcheck:

```
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:8000/health"]
  interval: 30s
  timeout:  10s
  retries:  3
```

Configuration Parity

- Same Docker image across Dev, Test, and Prod environments
- Behaviour only changes through environment variables and injected JSON
- app.json is the only file changed to update network policies
- No code changes required to add or remove service relationships

11 / TECH STACK — FULL DEPENDENCY OVERVIEW

BACKEND / API

FastAPI

High-performance Python REST framework; async-ready, auto-docs

Pydantic

Data validation via Python type hints; drives OpenAPI schema

Uvicorn

ASGI server; handles concurrent requests with uvloop

PyYAML

Serialises Python dicts → YAML for the NetworkPolicy output

INFRASTRUCTURE

Docker

Containerises the app; python:3.11-slim base for security

Docker Compose

Orchestrates service, ports, env vars, and healthcheck

Kubernetes

Target platform; consumes generated NetworkPolicy YAML

CI/CD & QUALITY

GitHub Actions

Pipeline runner; ubuntu-latest, triggers on push/PR

pytest

Unit and integration test framework; httpx for async API tests

flake8

PEP 8 linter; E9/F63/F7/F82 — syntax errors & undefined names

httpx

Async HTTP client used in CI integration tests

12 / REPOSITORY STRUCTURE

Following professional DevOps directory standards — clear separation of concerns across every layer.

Devops_project/	Repository root
├── .github/workflows/ci-cd.yml	GitHub Actions pipeline definition
├── docs/ARCHITECTURE.md	Technical architecture documentation
├── docs/USAGE.md	Configuration & usage guide
├── infrastructure/docker/Dockerfile	Multi-stage container build spec
├── infrastructure/docker/docker-compose.yml	Service orchestration & healthcheck
├── infrastructure/kubernetes/app.json	IaC input — service network intent
├── src/main.py	FastAPI app — routing, auth, endpoints
├── src/logic.py	Core engine — AppSpec → NetworkPolicy
├── src/tests/test_logic.py	Unit & integration test suite
├── requirements.txt	Python dependency manifest
├── .gitignore	Excludes __pycache__, .venv, *-policy.yaml
├── LICENSE	MIT License — Arya Duttagupta 2026

13 / HOW TO RUN — LOCAL · DOCKER · CURL

1. Clone & Install

```
git clone https://github.com/unpetrifiedstone2005/Devops_project.git
cd Devops_project
pip install -r requirements.txt
```

2. Run Locally (Dev Mode)

```
python -m uvicorn src.main:app --reload
# Available at: http://127.0.0.1:8000
# Swagger UI:  http://127.0.0.1:8000/docs
```

3. Run with Docker Compose

```
docker compose -f infrastructure/docker/docker-compose.yml \
  up --build
# Serves on port 8000 with health checks
```

4. Test with curl

```
curl -X POST http://localhost:8000/generate \
  -H "X-API-KEY: devops-internal-secret" \
  -H "Content-Type: application/json" -d @app.json
```

5. Run Tests

```
python -m pytest src/tests/ -v
# Runs unit + integration tests
# All tests must pass in CI pipeline
```

6. Troubleshooting

```
403 → Check X-API-KEY header value
422 → Ensure 'name' and 'labels' fields present
404 → URL must end with /generate
ModuleNotFoundError → Use python -m uvicorn
```

PROJECT SUMMARY

14 / What This Project Delivers

- Automated Kubernetes NetworkPolicy YAML generation from a simple JSON spec
- 7-stage GitHub Actions CI/CD: lint → test → Docker build → Compose validate
- Zero Trust networking enforced at generation time — deny-all by default
- Containerised on python:3.11-slim with healthchecks and restart policies
- Full Infrastructure as Code: app.json as the single source of truth
- Structured logging and /health probe ready for orchestrator integration
- Secure REST API with X-API-KEY auth, Pydantic validation, and Swagger UI
- Clean repository layout following professional DevOps directory standards